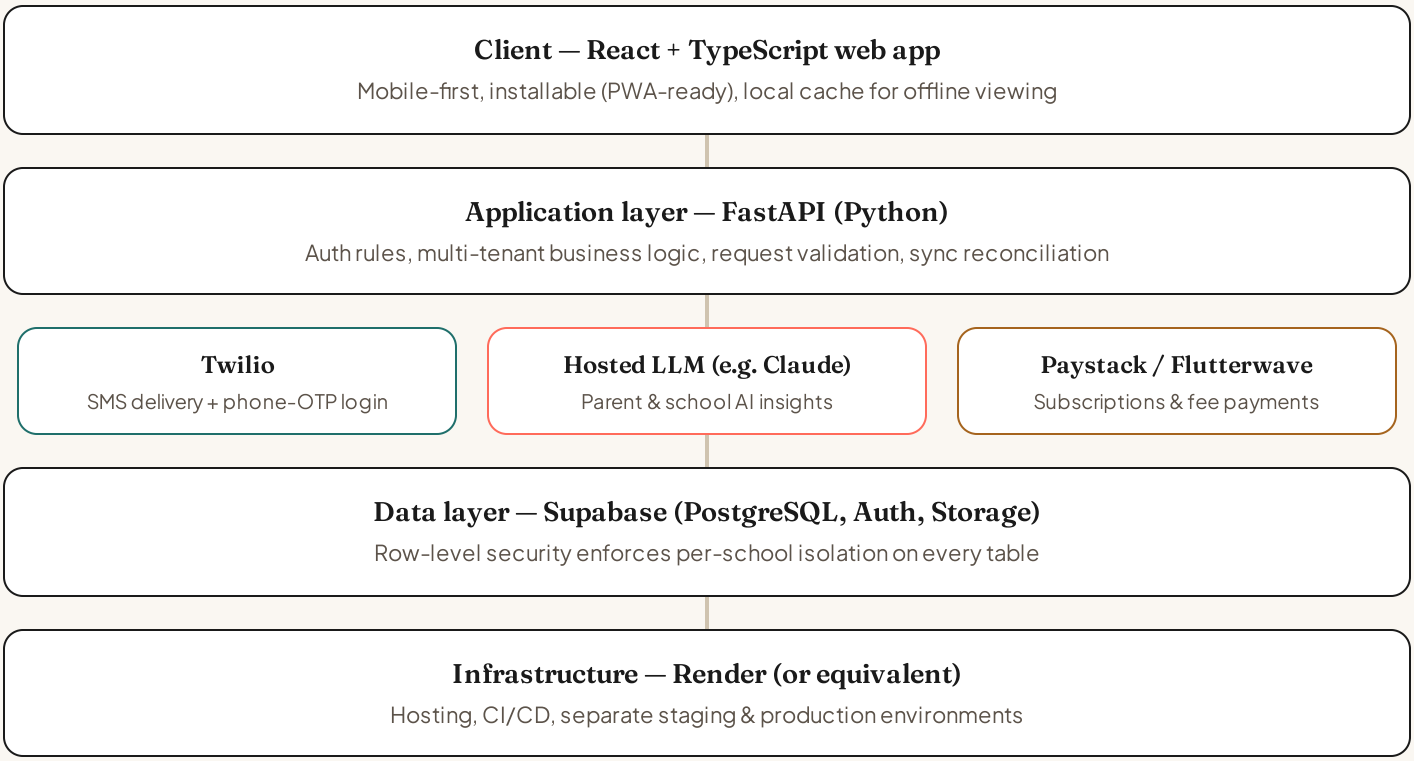
REFERENCE DOCUMENT · ASSUMES PHASES 1–3 COMPLETE

Full-build technical architecture

A one-look snapshot of how Kithnest is architected once every phase is built and live

This is a snapshot of Kithnest's technical architecture assuming all three build phases are complete: a live, multi-tenant product serving real schools, with real messaging, AI-driven insight, and billing. It's a companion to the longer *Technical & Product Overview*, which explains the reasoning behind each choice in more depth.



Read top to bottom: a parent or admin's device talks only to the application layer, never directly to the database or to Twilio/AI/payment providers. Every integration is reachable only through FastAPI, which is also where school-to-school data isolation is enforced before it ever reaches Supabase.

REFERENCE DOCUMENT · ASSUMES PHASES 1–3 COMPLETE

The stack, layer by layer

Every technology choice behind the architecture on the previous page

FRONTEND

- **Framework:** React + TypeScript, built with Vite
- **Design system:** Tailwind CSS, custom colour/type/spacing tokens
- **Animation:** Framer Motion, used sparingly and purposefully
- **Delivery:** installable, PWA-ready web app with offline shell caching

BACKEND & DATA

- **Application layer:** FastAPI (Python)
- **Database:** PostgreSQL, managed via Supabase
- **Auth:** Supabase Auth, phone number + one-time code, role-based (parent / teacher / admin)
- **File storage:** Supabase Storage (photos, attachments)
- **Multi-tenancy:** PostgreSQL row-level security, per school
- **Offline sync:** local cache with reconnect reconciliation

MESSAGING

- **Provider:** Twilio, SMS delivery + OTP login
- **Pattern:** wrapped behind an internal service so the provider can be swapped without touching product logic

AI & INSIGHTS

- **Provider:** hosted large language model (e.g. Claude), called server-side
- **Parent-facing:** plain-language progress summaries over time
- **School-facing:** engagement & performance risk flags
- **Data boundary:** scoped per school, same isolation as core data; NDPR-aware given this is children's data

BILLING

- **Provider:** Paystack or Flutterwave
- **Scope:** school subscription tiers + digital fee payments
- **Isolation:** dedicated billing service boundary, decoupled from core product logic

HOSTING & DEVOPS

- **Platform:** Render (or an equivalent — Railway / Fly.io)
- **Environments:** separate staging and production
- **Pipeline:** automated build/test on push, deploy on merge to main

SECURITY & COMPLIANCE

- **Tenant isolation:** enforced at the database level, not just in the interface
- **Data protection:** NDPR-aware handling throughout, given the product processes minors' data
- **Auth:** one-time-code login for parents; no stored passwords to leak

DATA MODEL (CORE ENTITIES)

- **School** → Classes → Pupils → Parents
- Many parents can map to one pupil; one parent can map to many pupils/schools
- Workload and Notification entities, each tied to a class or pupil group

This reflects real, considered decisions rather than placeholders, but nothing here is contractually fixed this early. See the *Technical & Product Overview* document for the reasoning behind each choice, phase by phase.